

METADRON CAPITAL — COMPLETE ARCHITECTURE DNA

MASTER SIGNAL PIPELINE (Execution Order)



LAYER 1: DATA INGESTION — UniverseEngine + OpenBB Data

UniverseEngine (`engine/data/universe_engine.py`)

- **4-run scan:** SP500(~500) + SP400(~400) + SP600(~600) + ETF/FI(~70) = ~1,600 securities
- **GICS 4-tier hierarchy**
- **11 GICS Sector ETFs:** XLE, XLB, XLI, XLY, XLP, XLV, XLF, XLK, XLC, XLU, XLRE
- **26 Relative Value pairs** (e.g., XLK/XLE, XLF/XLU, XLY/XLP)
- **GSIB basket:** JPM, GS, MS, BAC, C, WFC + 5 international banks
- **Macro tickers:** SPY, QQQ, IWM, TLT, GLD, USO, UUP, HYG, LQD, VXX
- **Methods:** `get_by_sector()` , `get_rv_pairs()` , `get_sectors()` , `get_gsib_basket()`

Data Source Architecture — Dual-Mode (Live + Historical)

```
MARKET HOURS (09:30-16:00 ET):
  Price Data:      IBKR real-time (via ib_insync reqMktData, ~100-500ms latency)
  Quote Cache:     5-second TTL per ticker
  Fallback:        OpenBB if IBKR quote unavailable
  OHLCV/History:   OpenBB (signals, quant strategies, alpha optimizer)
  FRED/Macro:      OpenBB (openbb-fred provider)
  Execution:       IBKR native algo orders (TWAP/VWAP/Adaptive/Market)

AFTER HOURS / OVERNIGHT:
  All Data:        OpenBB (34+ providers) – backtesting, model retraining
  Historical:       OpenBB for walk-forward validation, Monte Carlo, scenario analysis
  FRED/SEC/CBOE:   OpenBB econometric providers

DATA SOURCE INDICATOR:
  The live dashboard and frontend must display the active data source:
  [LIVE] IBKR real-time      – during market hours
  [EOD] OpenBB historical    – after hours / backtesting
  This should be interchangeable on command via the dashboard or config.
```

OpenBB Data (`engine/data/openbb_data.py` , re-exported via `yahoo_data.py`)

- **Unified data layer:** All data via OpenBB (34+ providers) — single API surface
- **Provider hierarchy:** Configurable per-function (default varies: fmp, polygon, tiingo, fred)
- **IBKR as data source:** During market hours, price data comes from IBKR via `ib_insync reqMktData()` with 5-second quote cache. Historical data and FRED/macro always via OpenBB.
- `get_adj_close(tickers, start, end)` → Adjusted close prices
- `get_returns(tickers, start, end)` → Daily log returns
- `get_prices(tickers, start)` → Full OHLCV data
- `get_macro_data(start)` → VIX, S&P 500, 10Y Yield, 5Y Yield, Gold, HY Corporate, IG Corporate
- `get_market_stats()` → SPY return, VIX level, current volatility

LAYER 2: SIGNAL PROCESSING — MacroEngine

MacroEngine (`engine/signals/macro_engine.py`)

Purpose: Regime classification + GMTF money velocity analysis

Regime Classification

```
MarketRegime: BULL | BEAR | TRANSITION | STRESS | CRASH
CubeRegime:   TRENDING | RANGE | STRESS | CRASH
```

Classification logic: - VIX > 35 → CRASH - VIX > 28 AND SPY_3M < -10% → STRESS - SPY_1M > 3% AND SPY_3M > 5% → BULL - SPY_1M < -3% AND SPY_3M < -5% → BEAR - Else → TRANSITION

GMTF (Global Monetary Tension Framework)

SDR basket weights (IMF 2022): USD=0.4338, EUR=0.2931, CNY=0.1228, JPY=0.0759, GBP=0.0744

Base tension: $\theta = (M2/GDP) \times (1+unemployment) \times GDP_growth$

4 Non-linear gamma multipliers: | Gamma | Trigger | Multiplier | |-----|-----|-----| | $\Gamma_Liquidity$ | M2 growth > 2× GDP growth | 1 + sigmoid × 1.5 | | Γ_FX | USD yield spike > 25bps | 1 - sigmoid × 0.5 | | Γ_Wage | Unemployment < 4.2% | 1 + sigmoid × 2.0 | | $\Gamma_Reserve$ | USD share < 57% | 1 + sigmoid × 1.5 |

Sigmoid: $\sigma(x) = 1 / (1 + \exp(-15 \times (x - threshold)))$

Aggregation: $GMTF = \Sigma(SDR_weight \times \theta \times \Gamma_L \times \Gamma_W \times \Gamma_FX \times \Gamma_S)$, smoothed with 5-day rolling mean

Sub-Modules (7 total)

1. **MoneyVelocityModule** — Fisher V=GDP/M2, credit impulse, TED spread, SOFR tracking
2. Liquidity Score 0-100: velocity(25%) + credit_impulse(20%) + TED(20%) + VIX(20%) + yield_curve(15%)
3. **SectorRanker** — Macro-adjusted momentum ranking
4. Composite = $Sharpe \times regime_multiplier \times (1 + relative_strength \times 0.5)$
5. Blended momentum: 50% 3M + 30% 1M + 20% 6M
6. Regime multipliers: BULL cyclicals ×1.25, STRESS defensives ×1.4
7. Factor rotation: MOMENTUM | VALUE | QUALITY | DEFENSIVE | NEUTRAL
8. **CarryToVolatility** — G10 FX carry-to-vol signals
9. $CtV = carry_spread / realised_vol(FX)$
10. Gate: CtV > 0.5 AND no stop-loss active
11. Stop-loss: vol spike > 2σ in 1 day
12. SDR-weighted aggregate CtV
13. **RegimeTransitionDetector** — Markov transition with hysteresis (3 days)

14. Base transition probabilities (5×5 matrix)
15. Confidence = 60% frequency + 40% base probability
16. **YieldCurveAnalyzer** — 2s10s, 3m10y, term premium, real rate
17. Curve shapes: STEEP | FLAT | INVERTED | BEAR_FLAT | BULL_STEEP
18. Recession probability via NY Fed probit: $P = \Phi(-0.5333 - 0.6330 \times \text{spread_3m10y})$
19. **CreditPulseMonitor** — HY-IG spread, z-score, credit impulse
20. Thresholds: NORMAL(<3.0) | ELEVATED(3.0-4.5) | STRESS(4.5-6.0) | CRISIS(>8.0)
21. **MacroFeatureBuilder** — 50+ features for ML
22. VIX(5) + Equity(8) + Yield(8) + Credit(5) + Commodity(6) + Momentum(8) + Volatility(5) + Regime(7+)

Output: **MacroSnapshot**

```
regime, vix, spy_return_1m, spy_return_3m, yield_10y, yield_2y,
yield_spread, credit_spread, gold_momentum, sector_rankings,
gmtf_score, money_velocity_signal, cube_regime
```

LAYER 2: SIGNAL PROCESSING — MetadronCube

MetadronCube (`engine/signals/metadron_cube.py`)

Purpose: $C(t) = f(L_t, R_t, F_t)$ — 10-layer intelligence tensor

10 Layers

#	Layer	Function	Output
0	FedPlumbingLayer	SOFR, HY spreads, M2V, TGA, ON-RRP, SOMA	Fed plumbing state
1	LiquidityTensor	Fed → PD → GSIB → Shadow bank reserve routing	$L(t) \in [-1, +1]$
2	ReserveFlowKernel	TVP: $\Delta \text{Reserves} \rightarrow \Delta \text{Sector } \beta \text{ at } t+1..t+10$	Sector beta projections
R	RiskStateModel	VIX + realized vol + credit + skew	$R(t) \in [0, 1]$
F	CapitalFlowModel	Sector momentum, leader/laggard, rotation	F(t) flow state
4	RegimeEngine	HMM+RL → 4 regimes	TRENDING/RANGE/STRESS/CRASH
G	GateZAllocator	5-sleeve capital allocation	Sleeve weights
E	GateLogic	4-gate entry scoring	Entry scores
K	KillSwitch	Auto-derisking triggers	Kill signal
C	FCLPLoop	Daily recalibration	Updated allocations

Regime Parameters (95% Alpha Target)

Regime	Max Leverage	β Cap	β Burst	Equity %	Hedge %	Crash Floor	θ Budget/day
TRENDING	3.0×	0.65	0.70	55%	5%	$\geq +25\%$	0.15%
RANGE	2.5×	0.45	0.55	40%	12%	$\geq +25\%$	0.10%
STRESS	1.5×	0.15	0.20	20%	30%	$\geq +25\%$	0.05%
CRASH	0.8×	-0.20	-0.10	5%	50%	$\geq +25\%$	0.02%

Portfolio Allocation Mix (Credit-Aware)

Deployment target: 95% of NAV + leverage (5% dry powder cash reserve)

No mega cap restrictions – all market caps eligible.

SLEEVE	ALLOCATION	CREDIT TIER	NOTES
IG Equities	40%	A/B (IG)	All caps, quality names
Options	25%	Mixed	10% IG + 10% HY + 5% Distressed
Bond/Commodity ETFs	10%	–	TLT, GLD, USO, HYG, LQD, etc.
HY Equities	10%	C/D (HY)	BB-B rated, leveraged
Distressed Equity	10%	E/F (Distressed)	Fallen angels, recovery plays
Cash (Dry Powder) reserve	5%	–	Never deployed, buying power

Credit routing uses `governance/credit_classification.json` (FMP Rating API) with Egan-Jones D/E proxy fallback from `security_analysis_engine.py`.

4-Gate Entry Logic

Gate	Weight	Function
G1 Flow/Headlines	20%	ETF creations + Tensor signal → shortlist
G2 Macro/Beta	25%	Kernel projections + rates/FX betas → filter
G3 Fundamentals	30%	Quality/ROIC/FCF + supply-chain penalty
G4 Momentum/Tech	25%	Breadth/leadership/gamma/vanna confirms

Kill-Switch Matrix

IF HY OAS +35bp AND VIX term flat/inverted AND breadth < 50%
THEN → auto $\beta \leq 0.35$, max tail spend

FCLP (Full Calibration Learning Protocol)

1. Ingest plumbing → 2. Recompute Tensor/Kernel → 3. Regime detect →
2. Gate scoring → 5. Risk pass → 6. Write allocations

Output: **CubeOutput**

```
regime, max_leverage, beta_cap, beta_burst, risk_budget,
liquidity_state, risk_state, flow_state, sleeve_allocation,
entry_scores, kill_switch_active, crash_floor
```

LAYER 2: ADDITIONAL SIGNAL ENGINES

SocialPredictionEngine (engine/signals/social_prediction_engine.py)

Source: AgentSimEngine (market microstructure simulation) - Runs agent-based market simulation (Kyle Lambda, HAM, order book dynamics) - Builds agent behavioral profiles - Computes topic-level sentiment (BULLISH/BEARISH/MOMENTUM/REVERSAL) - Maps topics → tickers via **TOPIC_TICKER_MAP** - Detects narrative regime (trending/reversing/stable) - **Output:** **SocialSnapshot** → feeds into Tier-6 of ML Vote Ensemble

SecurityAnalysisEngine (engine/signals/security_analysis_engine.py)

Reference: Security Analysis 7th Edition (Graham, Dodd, Klarman) - **Pipeline Position:** Stage 3.1 — after MetadronCube (L2), before PatternDiscovery (L2/3.2) - **Top-Down Analysis** (Part I): - Interest rate as master variable → rate-adjusted max P/E = $\min(20, 1/\text{treasury}_{10y})$ - CAPE/Shiller cyclically adjusted P/E → expected 10yr return = $1/\text{CAPE}$ - Equity Risk Premium = $\text{earnings_yield} - \text{treasury}_{10y}$ - Speculative component = $(\text{market_pe} - \text{max_investment_pe}) / \text{market_pe}$ - Credit cycle indicators (HY spreads → implied default probability) - Market regime: DEEPLY_UNDERVALUED → EXTREMELY_OVERVALUED - **Bottom-Up Analysis** (Parts II-V): - Graham Number = $\sqrt{(22.5 \times \text{EPS} \times \text{BVPS})}$ - NCAV = Current Assets - ALL Liabilities (net-net screen) - 5-method intrinsic value estimation (Graham Number, Earning Power, Max P/E, NCAV, DCF-lite) - Margin of Safety = $(\text{IV} - \text{Price}) / \text{IV} \geq 33\%$ - Normalized EPS (5-10yr average), earnings stability ratio - ROIC-WACC spread (7th Ed. primary metric), DuPont ROE decomposition - Owner earnings (Buffett/7th Ed.), economic profit/EVA - 8-test investment grading (STRONG_INVESTMENT → AVOID) - Composite score: 25% MoS + 20% earnings + 15% balance sheet + 15% ROIC + 10% coverage + 10% valuation + 5% top-down - **Comparative Analysis:** Peer group relative metrics, Graham's 50% exchange premium rule - **Output:** SecurityAnalysisResult → feeds Tier-5 of MLVoteEnsemble + 12 alpha features for ML walk-forward

DistressedAssetEngine (engine/signals/distressed_asset_engine.py)

Reference: FinancialDistressPrediction, financial-distressed-repo, sophisticated-distress-analysis, Security Analysis 7th Edition (Mielle, Marks chapters) - **5-model ensemble + Graham-Mielle framework:** 1. Altman Z-Score: $Z = 1.2 \times \text{WC}/\text{TA} + 1.4 \times \text{RE}/\text{TA} + 3.3 \times \text{EBIT}/\text{TA} + 0.6 \times \text{MV}/\text{TL} + 1.0 \times \text{S}/\text{TA}$ 2. Merton KMV: Distance-to-default via option pricing on firm equity 3. Ohlson O-Score: Logistic regression on financial ratios 4. Zmijewski: Probit model on financial ratios 5. ML GBM: Gradient boosted machine on combined features - Fallen angel detector (investment grade → high yield transitions) - LGD (Loss Given Default) estimator - Kelly-sized opportunity generation - **Signals:** DISTRESS_FALLEN_ANGEL, DISTRESS_RECOVERY, DISTRESS_AVOID

CVREngine (engine/signals/cvr_engine.py)

Reference: ai-hedgefund, Financial-Data - **5-model CVR valuation:** 1. Binary Option (Black-Scholes digital) 2. Barrier Option (knock-in/knock-out) 3. Milestone Tree (decision tree valuation) 4. Monte Carlo (path simulation) 5. Real Options (expansion/abandonment value) - Liquidity + credit adjustments - 4 live CVR instruments tracked - **Signals:** CVR_BUY, CVR_SELL

EventDrivenEngine ([engine/signals/event_driven_engine.py](#))

Reference: TradeTheEvent, CTA-code, quant-trading - **12 event categories:** M&A/merger arb, PEAD (post-earnings drift), spinoffs, restructuring, activist, regulatory, share buyback, secondary offering, index rebalance, management change, dividend, catalyst - Mitchell-Pulvino M&A arbitrage model - SUE (Standardized Unexpected Earnings) PEAD model - Kelly-sized positions - 10 live event opportunities tracked - **Signals:** EVENT_MERGER_ARB, EVENT_PEAD_LONG, EVENT_PEAD_SHORT, EVENT_CATALYST

ContagionEngine ([engine/signals/contagion_engine.py](#))

- **21-node graph:** 6 GSIB banks + 11 GICS sectors + 4 macro assets
- Pure adjacency-matrix (no NetworkX)
- **7 shock scenarios:** BANK_RUN, CREDIT_CRUNCH, SOVEREIGN_CRISIS, SECTOR_ROTATION, LIQUIDITY_FREEZE, COMMODITY_SHOCK, CORRELATION_SPIKE
- Multi-step propagation with dampening
- Portfolio contagion risk scoring

StatArbEngine ([engine/signals/stat_arb_engine.py](#))

- Medallion-style mean reversion + cointegration pairs
- Factor residual extraction ($\Sigma\beta \approx 0$)
- Z-score entry/exit thresholds
- **Signals:** RV_LONG, RV_SHORT, MICRO_PRICE_BUY, MICRO_PRICE_SELL

LAYER 3: ML/AI — AlphaOptimizer

AlphaOptimizer ([engine/ml/alpha_optimizer.py](#))

Purpose: Walk-forward ML alpha prediction + mean-variance portfolio construction

Pipeline

1. **Feature engineering** (20+ features): market mean/vol, momentum (1w/1m/3m), RSI-14, MACD (12/26/9), Bollinger Band width/position, ATR-14, skewness, kurtosis, cross-sectional dispersion, momentum acceleration, vol ratio
2. **Walk-forward regression:** Linear → Ridge → XGBoost/GBR fallback
3. **EWMA covariance:** $\lambda=0.94$, span=60 days
4. **Mean-variance optimization:** SLSQP with turnover constraints (max 50%)
5. **Quality tier classification** (A-G)

Quality Tiers

Tier	Min Sharpe	Min Momentum
A	≥ 2.0	$\geq 15\%$
B	≥ 1.5	$\geq 10\%$
C	≥ 1.0	$\geq 5\%$
D	≥ 0.5	$\geq 0\%$
E	≥ 0.0	$\geq -5\%$
F	≥ -0.5	$\geq -10\%$
G	Below all	Below all

Extended Components

- **CAPMAAlphaExtractor**: Jensen's alpha, multi-factor decomposition (market, size, value, momentum, quality)
- **FactorLibrary**: 50+ factors across 6 categories (momentum×10, value×10, quality×10, volatility×10, technical×10, fundamental×10)
- **WalkForwardOptimizer**: Rolling window with XGBoost/Ridge fallback
- **MeanVarianceOptimizer**: EWMA cov + turnover + position limits + risk budgeting
- **AlphaDecayModel**: Half-life estimation, projected alpha at 5d/20d
- **TransactionCostModel**: Spread + market impact + commission
- **FeatureImportanceTracker**: Feature importance over time

Output: **AlphaOutput**

```
signals (list[AlphaSignal]), optimal_weights, expected_annual_return,
annual_volatility, sharpe_ratio, max_drawdown, rebalance_cost, alpha_predictions
```

SocialFeatureBuilder (`engine/ml/social_features.py`)

- Sentiment momentum (EMA fast/slow, MACD, z-score)
- Engagement velocity + acceleration
- Consensus strength + influence Gini coefficient
- Composite `social_alpha` signal → feeds into ML models

PatternRecognition (`engine/ml/pattern_recognition.py`)

- Candlestick patterns (hammer, engulfing, doji, etc.)
- Chart patterns (head-and-shoulders, double top/bottom)
- Statistical anomaly detection

Backtester (`engine/ml/backtester.py`)

- Walk-forward backtesting
- Monte Carlo simulation (10,000 paths)
- Scenario engine (historical stress scenarios)

UniverseClassifier (`engine/ml/universe_classifier.py`)

Purpose: XGBoost 4-model soft-voting ensemble for quality tier classification (A-G) - **4 models:**

GaussianNB (15%), GradientBoosting (25%), RandomForest (25%), XGBoost (35%) - **T3.1 hyperparams:**

`n_estimators=120`, `max_depth=6`, `lr=0.1`, `gamma=0`, `reg_lambda=10` - **16 features:** Sharpe, momentum (3M/6M), volatility, max drawdown, ROE, D/E, interest coverage, current ratio, revenue growth, earnings stability, FCF yield, gross margin, Piotroski F, Altman Z, beta - **CreditQualityClassifier:** 6-factor weighted model

(AAA-D credit ratings) - Interest coverage (25%), D/E ratio (20%), ROE (15%), earnings stability (15%),

Altman Z (15%), FCF yield (10%) - **Reconciliation engine:** top-down vs bottom-up tier divergence detection

- Rising stars: observable tier better than fundamental - Fallen angels: fundamental tier better than

observable - Feeds credit scores into T10 of MLVoteEnsemble

Credit Rating Architecture (Two-Layer)

Layer 1 – FMP Credit Rating API (`governance/classify_credit.py`)

Source: FMP `/api/v3/rating/{symbol}` – composite credit rating

Model: Proprietary (DCF score, ROE score, ROA score, D/E score, P/E, P/B)

Output: `governance/credit_classification.json`

Grades: AAA, AA, A, BBB (IG) | BB, B, CCC, CC, C, D (HY)

Use: Universe classification, risk tier assignment, portfolio constraints

Layer 2 – Egan-Jones D/E + CR Proxy (`engine/signals/security_analysis_engine.py`)

Source: Bottom-up fundamental analysis (Graham-Dodd framework)

Model: Debt-to-equity + current ratio thresholds

Output: `BottomUpScore.egan_jones_tier` (A-F) + `egan_jones_ig` (bool)

Tiers: A (`D/E<0.5`, `CR>1.5`) | B (`D/E<1.0`, `CR>1.2`) | C-F (increasing leverage)

Use: Per-security fundamental measure within the 8-test Graham grading system
Feeds into investment grade classification alongside interest coverage, equity cushion ratio, and accrual quality checks

These layers are complementary:

- FMP ratings are used at the portfolio/universe level (governance)
- Egan-Jones proxy is used at the per-security analysis level (bottom-up)
- Both feed into the CreditQualityClassifier and T10 of MLVoteEnsemble

ModelEvaluator (`engine/ml/model_evaluator.py`)

- Per-class precision/recall/F1 scoring
- Confusion matrix with tier-aware distance weighting
- Walk-forward evaluation metrics

DeepLearningEngine (`engine/ml/deep_learning_engine.py`)

Purpose: Pure-numpy PPO agent for trading decisions - 50-feature state vector (no external ML framework dependency) - Actor-Critic architecture with GAE (Generalized Advantage Estimation) - Trading environment with position management and transaction costs

ML Model Bridges (`engine/ml/bridges/`)

Signal bridges that produce SignalType values for MLVoteEnsemble: - **FinRLBridge**: FinRL deep RL framework adapter → DRL_AGENT signals - **NvidiaTFTAdapter**: NVIDIA Temporal Fusion Transformer → TFT signals - **MonteCarloBridge**: Monte Carlo simulation → MC signals - **StockPredictionBridge**: Stock prediction model → ML_AGENT signals - **DeepTradingFeatureBuilder**: Deep trading feature engineering - **KServeAdapter**: KServe ML model serving → remote inference

All bridges follow: try/except on imports, pure-numpy fallbacks, graceful degradation.

LAYER 4: PORTFOLIO — BetaCorridor + DecisionMatrix

BetaCorridor (`engine/portfolio/beta_corridor.py`)

Purpose: Manage portfolio beta within 7%-12% return corridor

Core Parameters

```
ALPHA = 2% (secular alpha headstart)
R_LOW = 7%, R_HIGH = 12% (Gamma Corridor)
BETA_MAX = 2.0, BETA_INV = -0.136 (hedge floor)
EXECUTION_MULTIPLIER = 4.7
VOL_STANDARD = 0.15 (thesis standard 15%)
VaR ≤ $0.30M (95%/1-day) on $20M NAV
```

Corridor Function (piecewise linear)

```
Rm < R_LOW → β = BETA_INV (short bias / hedge)
R_LOW ≤ Rm ≤ R_HIGH → β = linear(0 → BETA_MAX)
Rm > R_HIGH → β = BETA_MAX (full throttle)
```

Vol Regime Classifier

Regime	Percentile	β Multiplier	β Cap
LOW_VOL	<25th	1.20×	2.0
NORMAL	25th-75th	1.00×	2.0
ELEVATED	75th-95th	0.65×	1.0
CRISIS	>95th	0.30×	0.3

Beta Smoothing Pipeline

Raw target → EMA($\alpha=0.3$) → Anti-whipsaw(2% threshold) → Rate limiter($\pm 0.25/\text{cycle}$) → Kalman filter

DecisionMatrix ([engine/execution/decision_matrix.py](#))

Purpose: 4-gate cross-asset quality filter + Kelly sizing + ABU beta management. Integrated into AlphaOptimizer pipeline — every trade passes through all 4 gates. All gates are asset-agnostic (equities, options, futures, ETFs treated uniformly).

4 Approval Gates (binary pass/fail)

Gate	Weight	Threshold	Function	ML Tiers
FUNDAMENTALS	40%	0.45	Quality, ROIC, FCF, Graham-Dodd, credit, earnings + regime quality modifier	T1, T5, T7, T9, T10
FLOW_HEADLINES	20%	0.35	ETF flow, news sentiment, sector rotation	T6, T8
MACRO_REGIME	20%	0.40	Direction alignment (TRENDING/RANGE/STRESS/CRASH × long/short), VaR headroom, drawdown	T3, T4
MOMENTUM	20%	0.35	RSI, MACD, breakout, cross-asset momentum	T1, T2

MIN_COMPOSITE_SCORE = 0.55 (must pass to approve trade) **FUNDAMENTALS is the critical gate** — must pass independently regardless of composite

Regime Alignment

Regime	Long Modifier	Short Modifier
TRENDING	1.0	0.3
RANGE	0.6	0.6
STRESS	0.3	0.8
CRASH	0.1	1.0

KellySizer

```
Standard: f* = (p × b - q) / b
  where p = win_prob, b = win/loss ratio, q = 1-p
Aggressive: f_agg = f* × 1.5
Vol-scaled: f_final = f_agg × (VOL_STANDARD / vol)
Capped at: MAX_SINGLE_POSITION_PCT = 20%
```

AlphaBetaUnleashed (1-minute cadence beta)

```
Rm_adjusted = Rm_realized + macro.rm_adjustment
target_beta = corridor_fn(Rm_adjusted) × 4.7 × vol_adj
MES_hedge_beta = target_beta - sleeve_beta
```

LAYER 5: EXECUTION ENGINE

ExecutionEngine (`engine/execution/execution_engine.py`)

Purpose: Full pipeline orchestrator + ML vote ensemble + risk gates

ML Vote Ensemble (10 tiers, each votes ±1)

Tier	Name	Weight	Logic
1	Pure-numpy 2-layer net	1.0	Feature → hidden(20) → output, sigmoid activation
2	Momentum/mean-reversion	1.2	Mom_21d > 0 → +1; z-score_63d < -2 → +1 (mean revert)
3	Volatility regime	0.8	Vol_21d < Vol_63d → +1 (vol compression = bullish)
4	Monte Carlo	0.9	ARIMA-like drift + noise → direction vote
5	Quality tier	1.1	SecurityAnalysis grade (STRONG_INVESTMENT/INVESTMENT → +1) + alpha quality fallback
6	MiroFish agent sim (market microstructure)	1.0	Agent consensus (Kyle Lambda, HAM) → ±1
7	Distress	0.9	DistressedAssetEngine signals → ±1
8	Event-driven	1.0	EventDrivenEngine signals → ±1
9	CVR	0.7	CVREngine valuation signals → ±1
10	Credit quality	0.9	UniverseClassifier CQS > 0.7 → +1; < 0.3 → -1

Final vote: Weighted sum of 10 tiers → [-10, +10] **Minimum edge:**

`effective_min_edge = 2.0 + max(0, -vote_score)` bps

Deep Trading Features

- **MicroPriceEngine:** Bid/ask estimation from OHLCV, order flow imbalance, urgency score
- **CrossAssetMonitor:** SPY-TLT, SPY-GLD, SPY-UUP correlations → risk-on/risk-off score
- **DeepTradingFeatures:** 15+ features per ticker (multi-horizon momentum, vol regime, z-score, skew, kurtosis)

Risk Gate Manager (8 gates, all must pass)

Gate	Limit	Description
G1 Position Size	10% NAV	Single position limit
G2 Sector Concentration	30% NAV	Sector exposure limit
G3 Daily Loss	3% NAV	Daily loss circuit breaker
G4 Gross Exposure	250%	Gross leverage limit
G5 Net Exposure	150%	Net leverage limit
G6 Trade Count	100/day	Trade throttle
G7 Drawdown	10%	Max drawdown halt
G8 Cash Sufficiency	Trade value	Cash check for buys

PaperBroker (`engine/execution/paper_broker.py`)

Purpose: Live HFT opportunity-based paper portfolio execution - Simulated broker using OpenBB prices — continuously active throughout the day - **Constantly scanning** for alpha opportunities via signal pipeline - **5% daily compound target** (minimum) — once hit, risk dials down to retain gains - Risk dial-down tiers: AGGRESSIVE (pre-target) → MODERATE (target hit) → DEFENSIVE (target + buffer) - Tracks positions, P&L, NAV, cash, exposures with real-time dashboard hooks - MicroPriceModel: bid/ask estimation, order flow imbalance, time-of-day slippage - RiskLimiter: 6 pre-trade risk checks (position size, sector concentration, daily loss, exposure) - PerformanceTracker: Sharpe, drawdown, win rate by signal type, rolling analytics - **Live Dashboard:** Observable via `engine/monitoring/live_dashboard.py` when connected to internet - Supports: BUY, SELL, SHORT, COVER with full audit trail - Position tracking with sector tagging, reconciliation, CSV export

ExchangeCoreEngine (`engine/execution/exchange_core_engine.py`)

Purpose: Ultra-low-latency order matching (Python implementation of exchange-core concepts) - LMAX Disruptor-style ring buffer event processing (pre-allocated numpy arrays) - Limit order book with price-time priority matching - Market/Limit/Stop order types with L3 order book depth - Batch processing for multiple simultaneous orders - Latency tracking (simulated microsecond timestamps) - Routes through matching engine before PaperBroker fill

WonderTraderEngine (`engine/execution/wondertrader_engine.py`)

Purpose: CTA strategy execution + HFT micro-price engine - CTA trend-following signals (dual MA crossover, channel breakout, momentum) - Multi-timeframe analysis (1m, 5m, 15m, 1h, 4h bars) - Smart order routing (TWAP/VWAP splitting for large orders) - Execution quality scoring (slippage vs benchmark) - Dynamic stop-loss / take-profit management

OptionsEngine (`engine/execution/options_engine.py`)

- Black-Scholes pricing (calls + puts)
- Greeks: Delta, Gamma, Theta, Vega, Rho
- Volatility surface construction
- **$\theta + \Gamma$ Optimizer:** $\max(\theta + \Gamma) - c \times \text{Vega}$ for P4 sleeve allocation
- Strategy builder for hedging (collars, put-spreads, VIX calendars)

ConvictionOverride (`engine/execution/conviction_override.py`)

Tier	Confidence	Multiplier	Agents Required
CONTROLLED	90-95%	1.5×	1
AGGRESSIVE	95-98%	2.0×	2
MAXIMUM	>98%	2.0×+	3

LAYER 7: UNIFIED EXECUTION SURFACE

L7UnifiedExecutionSurface (`engine/execution/l7_unified_execution_surface.py`)

Fuses WonderTrader (micro-price + CTA + TWAP/VWAP routing), ExchangeCore (order matching), IBKRBroker (sole execution broker with native algo orders), and OptionsEngine (derivatives) into one continuous execution arm.

ALL tradeable products route through IBKR as the sole execution broker. ALL orders MUST enter through `L7UnifiedExecutionSurface.submit_order()` — never direct broker calls. Trade log records every generated order for reconciliation (generated vs executed). Fixed income, FX, and liquidity instruments are for research only — never executed.

Architecture

```

L7UnifiedExecutionSurface
├─ Continuous intraday loop (1-min heartbeat from live_loop_orchestrator)
├─ Multi-product router (equities, options, futures)
│   ├── Equity → WonderTrader micro-price → ExchangeCore → IBKR (TWAP/VWAP)
│   ├── Options → OptionsSizer (BS+MC+Kelly edge gate) → IBKR
│   └─ Futures → Beta corridor hedge → IBKR
├─ Unified order book (all products, all horizons)
├─ IBKRBroker (sole execution) + trade log (reconciliation)
├─ L7RiskEngine (10 gates, per-execution update)
├─ TransactionCostAnalyzer (per-trade decomposition)
├─ ExecutionLearningLoop (pattern identification)
├─ SlippageModel (pre-trade cost estimation)
└─ Prometheus metrics (17 gauges/counters/histograms)

```

10-Step Execution Flow

1. **Research-only guard** — reject FI/FX/credit instruments
2. **Product classification** — equity, option, or future
3. **Learning loop suggestion** — optimal routing from pattern library
4. **Pre-trade risk gates** — 10 gates must all pass
5. **Slippage estimation** — sqrt market impact model
6. **Product-specific path** — micro-price, OptionsSizer, beta corridor
7. **IBKR execution** — TWAP (>\$50K), VWAP, Adaptive, or Market algo
8. **Post-trade risk update** — risk state refreshed + Prometheus metrics
9. **TCA analysis** — spread, impact, timing, commission decomposition
10. **Learning loop recording** — EWMA pattern update + trade log for recon

Algo Routing (IBKR Native)

Routing	Trigger	IBKR Algo
TWAP	Notional >\$50K or SMART routing	<code>algoStrategy="Twap"</code> — server-side time-weighted splitting
VWAP	Explicit VWAP routing	<code>algoStrategy="Vwap"</code> — volume-weighted participation (max 25%)
Adaptive	Medium urgency	<code>algoStrategy="Adaptive"</code> — IBKR selects optimal strategy
Market	HIGH/CRITICAL urgency	Direct market/limit order

Risk Gates (10)

Gate	Limit	Description
G1	10% NAV	Single position limit
G2	30% NAV	Sector concentration
G3	3% NAV	Daily loss circuit breaker
G4	250%	Gross leverage
G5	150%	Net leverage
G6	100/day	Trade throttle
G7	10%	Max drawdown halt
G8	cash	Cash sufficiency
G9	20% NAV	Options delta exposure
G10	50% NAV	Futures notional

TCA Decomposition

Component	Model
Spread	Half bid-ask spread (calibrated per product)
Market Impact	$\text{sqrt}(\text{participation}) \times \text{volatility} \times \text{coefficient}$
Timing	Arrival-to-fill price drift
Commission	IBKR tiered (\$0.005/share equity, \$0.65/option, \$1.25/future)

Options Sizing (Edge-Gated)

Options ONLY proceed when mispricing is detected:

```
OptionsSizer (engine/execution/options_engine.py)
├─ Step 1: Black-Scholes theoretical price
├─ Step 2: Monte Carlo simulation (10K paths) → win prob + expected payoff
├─ Step 3: Fair value = avg(BS, MC) vs market price → edge in bps
├─ Step 4: Edge < 200bps → REJECTED (no allocation)
├─ Step 5: Kelly criterion sizes based on detected edge
├─ Step 6: Minimum 5 contracts – no token 1-2 positions
└─ Output: contracts, greeks, kelly_fraction, edge_bps
```

No market price available → REJECTED. No mispricing detected → REJECTED. Budget insufficient for 5 contracts → REJECTED.

Prometheus Metrics (17)

Metric	Type	Description
<code>l7_orders_total</code>	Counter	Total orders by product/side/algo
<code>l7_orders_filled</code>	Counter	Filled orders by product/algo
<code>l7_orders_rejected</code>	Counter	Rejected orders by reason
<code>l7_fill_latency_seconds</code>	Histogram	Fill latency distribution
<code>l7_slippage_bps</code>	Summary	Realized slippage by product
<code>l7_nav_usd</code>	Gauge	Current NAV
<code>l7_gross_leverage</code>	Gauge	Gross leverage ratio
<code>l7_net_leverage</code>	Gauge	Net leverage ratio
<code>l7_position_count</code>	Gauge	Active positions
<code>l7_daily_pnl_usd</code>	Gauge	Daily P&L
<code>l7_risk_level</code>	Gauge	0=NORMAL, 1=ELEVATED, 2=HIGH, 3=CRITICAL
<code>l7_kill_switch_active</code>	Gauge	Kill switch status (0/1)
<code>l7_twap_orders</code>	Counter	TWAP algo orders
<code>l7_vwap_orders</code>	Counter	VWAP algo orders
<code>l7_ibkr_connected</code>	Gauge	IBKR connection status (0/1)
<code>l7_tca_total_cost_bps</code>	Summary	TCA total cost per trade
<code>l7_tca_is_usd</code>	Summary	Implementation shortfall USD

Trade Log (Reconciliation)

Every order the platform generates is recorded in `logs/l7_execution/trade_log/`:

```
trade_log_YYYYMMDD.jsonl — one JSON line per order:
  order_id, ticker, side, quantity, product_type, signal_type,
  routing, arrival_price, micro_price, generated_at,
  broker_status (FILLED | NOT_EXECUTED | ERROR), broker_fill_price, broker_algo
```

`get_recon_summary()` → total_generated vs executed_on_broker vs not_executed vs errors

Execution Learning Loop

- **Intraday**: EWMA update of slippage/impact per context bucket
- **Daily**: Re-rank routing strategies

- **Weekly:** Decay old samples, refresh pattern weights
- **Monthly:** Prune stale patterns, recalibrate coefficients

Context buckets: ticker × product × signal × regime × time-of-day × volatility × order size

LAYER 6: AGENT ORCHESTRATION

SectorBots (`engine/agents/sector_bots.py`)

- **11 GICS sector micro-bots**, one per sector
- Each bot: analyzes sector momentum, generates signals, tracks accuracy
- Weekly scoring: 40% accuracy + 30% Sharpe + 30% hit rate
- Promotion/demotion system

ResearchBots (`engine/agents/research_bots.py`)

- **11 GICS research bots** with DNA hierarchy
- Daily research cycle per sector
- Intelligence reports with conviction levels
- Bot-to-bot communication for cross-sector signals

Agent Scorecard (`engine/agents/agent_scorecard.py`)

Rank	Requirements	Promotion
DIRECTOR	Sharpe >2.5, accuracy >85%	Top performer 8+ weeks
GENERAL	Sharpe >2.0, accuracy >80%	4 consecutive top weeks
CAPTAIN	Sharpe >1.5, accuracy >55%	Default
LIEUTENANT	Sharpe >1.0, accuracy >50%	Default
RECRUIT	Below thresholds	Demoted after 2 bottom weeks

GICS Sector Agents (`engine/agents/gics_sector_agents.py`)

- **11 GICS sector agents** with 8 scoring dimensions each
- Scoring: momentum, quality, value, growth, risk, sentiment, technical, macro alignment
- Sector-specific weight calibration
- Cross-sector signal correlation tracking

Agent Monitor (`engine/agents/agent_monitor.py`)

4-tier performance hierarchy: | Tier | Sharpe | Win Rate | Status | |-----|-----|-----|-----| | ELITE | >2.0
| >70% | Full autonomy | | STRONG | >1.5 | >60% | Standard operation | | DEVELOPING | >0.5 | >45% |
Restricted sizing | | UNDERPERFORM | <0.5 | <45% | Review + potential shutdown |

- Per-agent memory tracking via `tracemalloc`
- Performance decay detection
- Automatic promotion/demotion with hysteresis

Investor Personas (`engine/agents/investor_personas.py`)

12 investor persona agents, each with unique investment philosophy: - **Warren Buffett**: Intrinsic value, moat, long-term hold - **Charlie Munger**: Multi-disciplinary mental models - **Benjamin Graham**: Deep value, margin of safety - **Bill Ackman**: Activist + concentrated positions - **Ray Dalio**: All-weather, macro regime balancing - **Howard Marks**: Credit cycles, risk awareness - **Seth Klarman**: Deep value + special situations - **Peter Lynch**: Growth-at-reasonable-price (GARP) - **George Soros**: Reflexivity, macro momentum - **Stanley Druckenmiller**: Macro + growth conviction - **David Einhorn**: Forensic accounting, short selling - **Cathie Wood**: Disruptive innovation, hyper-growth

8 Core Analysis Agents: fundamentals, technicals, sentiment, risk management, macro, valuation, portfolio optimization, sector rotation

LIVE EXECUTION — IBKR BROKER INTEGRATION (SOLE BROKER)

Broker Hierarchy

```
IBKRBroker      → SOLE EXECUTION: Equities + Options + Futures (TWS/Gateway via
ib_insync)
Trade Log       → RECONCILIATION: Records all generated orders vs broker fills
PaperBroker     → BACKTESTING ONLY: Historical simulation (never used for live)
```

IBKRBroker (`engine/execution/ibkr_broker.py`)

Purpose: Sole execution broker — routes ALL orders through IBKR TWS/Gateway with native TWAP/VWAP algo support. No other broker is used for execution. All orders MUST route through `L7UnifiedExecutionSurface` → `IBKRBroker`. Direct broker calls are PROHIBITED.

Configuration

```
export IBKR_HOST=127.0.0.1      # TWS/Gateway host
export IBKR_PORT=7497          # 7497 = paper, 7496 = live
export IBKR_CLIENT_ID=1        # TWS client ID
export IBKR_PAPER_TRADE=True   # True = paper, False = live
```

Native Algo Orders

- **TWAP:** `place_twap_order()` — IBKR server-side time-weighted splitting (configurable duration)
- **VWAP:** `place_vwap_order()` — IBKR volume-weighted participation (max 25% of volume)
- **Adaptive:** `place_adaptive_order()` — IBKR auto-selects optimal algo (Patient/Normal/Urgent)
- **Market/Limit:** `place_order()` — standard order (PaperBroker-compatible interface)

Features

- **SDK:** `ib_insync` (official Python async wrapper for IBKR TWS API)
- **Commission:** \$0.005/share equity (min \$1), \$0.65/option, \$1.25/future
- **Futures:** ES, NQ, YM, RTY, VX, ZN, ZB, ZF, ZT — all tradeable
- **Position sync:** Live positions from IBKR account
- **Account sync:** NAV, cash, buying power, gross position value
- **Quote cache:** 5-second TTL
- **Retry logic:** 4 retries with exponential backoff (2s, 4s, 8s, 16s)
- **Algo fill polling:** `check_algo_fills()` — TWAP/VWAP may take minutes
- **Full audit trail:** All orders logged to `logs/ibkr_broker/`

MANDATORY WIRING RULES

Any AI model, CLI agent, or external tool that instantiates this system **MUST** follow these rules.

Validate with `python3 -m engine.wiring_manifest` before any execution.

- RULE 1: ALL orders MUST route through `L7UnifiedExecutionSurface.submit_order()`
NEVER call `IBKRBroker` directly. NEVER use raw HTTP requests to any broker API.
- RULE 2: ALL equity orders MUST pass through `WonderTrader` micro-price before IBKR.
- RULE 3: ALL options MUST go through `OptionsSizer` (BS + MC + Kelly).
REJECTED if no market price, no mispricing edge $\geq 200\text{bps}$, or < 5 contracts.
- RULE 4: ALL orders MUST pass `L7RiskEngine` 10-gate pre-trade check before execution.
- RULE 5: ALL fills MUST update `LearningLoop.record_signal_outcome()` after execution.
- RULE 6: ALL fills MUST generate TCA analysis via `TransactionCostAnalyzer`.
- RULE 7: `MetadronCube` `KillSwitch` overrides ALL allocation decisions.
When active, no new orders are submitted.
- RULE 8: `DecisionMatrix` `MIN_COMPOSITE_SCORE` = 0.55. Trades below this are REJECTED.
- RULE 9: `IBKRBroker` is the SOLE execution broker. No Alpaca, no Tradier, no raw API.
- RULE 10: Trade log records ALL generated orders for reconciliation. Every order the platform generates must be logged regardless of execution outcome.

TRADING AUTOMATION — LIVE LOOP ORCHESTRATOR

LiveLoopOrchestrator (`engine/live_loop_orchestrator.py`)

Purpose: 7-phase continuous heartbeat loop (1-minute cadence, 09:30–16:00 ET)

CONTINUOUS LOOP (1-min heartbeat)			
Phase 1	DATA	(every tick)	DataIngestion → UniversalPool
Phase 2	SIGNALS	(1-min)	Macro, Cube, Liquidity, Fund.
Phase 3	INTELLIGENCE	(5-min)	Alpha, ML ensemble, Agents
Phase 4	DECISION	(on signal Δ)	DecisionMatrix, BetaCorridor
Phase 5	EXECUTION	(on approval)	L7 → IBKR (TWAP/VWAP)
Phase 6	LEARNING	(continuous)	GSD, Paul, LearningLoop
Phase 7	MONITORING	(5-min)	Dashboard, HourlyCSV, Reports

Schedule

Time	Mode	Cadence	Actions
08:00–09:30	Pre-market	Full refresh	Overnight signals, SEC scan, macro update
09:30	Market open	Flush	Full pipeline execution, first trades
09:30–16:00	Intraday	1-min heartbeat	All 7 phases active
16:00	Market close	EOD	Reconciliation, learning snapshot, scorecard
16:00–20:00	After-hours	Reduced	Earnings scan, reduced frequency
20:00–08:00	Overnight	Batch	Backtesting, ML retraining, pattern evolution

Graceful Degradation

All imports wrapped in `try/except` (Rule 8). If any component fails: - System logs warning and continues with remaining engines - Missing engines produce empty signals (not crashes) - GSD/Paul plugins optional — system operates without them

LEARNING SYSTEM — GSD + PAUL DYNAMIC INTEGRATION

GSD Plugin (Gradient Signal Dynamics) (`intelligence_platform/plugins/gsd_paul_plugin.py`)

Purpose: Track signal momentum, convergence, divergence, and gradient decay across all engines

```

GSDPlugin
├── Signal Gradient Tracking
│   ├── Per-engine signal momentum (rolling gradient)
│   ├── Cross-engine convergence detection
│   ├── Divergence alerts (when engines disagree)
│   └── Gradient decay measurement (alpha half-life)
├── Adaptive Learning Rate
│   ├── Per-agent learning rate adjustment
│   ├── Regime-dependent rate scaling
│   └── Performance-weighted gradient updates
└── Integration Points
    ├── LiveLoopOrchestrator Phase 6 → GSDPlugin.update_gradients()
    ├── EnforcementEngine → proactive drift detection
    ├── DynamicAgentFactory → gradient-driven agent spawning
    └── PaulOrchestrator → gradient metrics feed Paul patterns

```

GSD Workflow Bridge ([intelligence_platform/plugins/gsd_workflow_bridge.py](#))

- **GSDPhase**: Individual phase within a gradient plan
- **GSDPlan**: Multi-phase gradient optimization plan
- **GSDTask**: Atomic gradient update task
- **GSDState**: Complete gradient state snapshot
- **GSDWorkflowBridge**: Connects gradient dynamics → engine weight adjustments

Paul Plugin (Pattern Awareness & Unified Learning) ([intelligence_platform/plugins/gsd_paul_plugin.py](#))

Purpose: Pattern memory, matching, evolution, and context-aware replay

```

PaulPlugin
├── Pattern Memory
│   ├── Discovered patterns stored as indexed library
│   ├── Pattern similarity matching (correlation-based)
│   ├── Pattern evolution tracking (mutation across regimes)
│   └── Context-aware pattern replay (regime-conditional)
├── Unified Pattern Library
│   ├── Cross-engine pattern sharing
│   ├── Pattern confidence scoring
│   ├── Pattern decay detection
│   └── Novel pattern discovery alerts
└── Integration Points
    ├── LiveLoopOrchestrator Phase 6 → PaulPlugin.store_pattern()
    ├── EnforcementEngine → pattern-driven consistency
    ├── DynamicAgentFactory → pattern-driven agent creation
    └── PaulOrchestrator → full pattern lifecycle management

```

AgentLearningWrapper (`intelligence_platform/plugins/gsd_paul_plugin.py`)

Purpose: Attaches GSD + Paul to any agent for continuous learning - Wraps any sector bot, research bot, or persona agent - Tracks per-agent gradient dynamics (signal accuracy over time) - Stores per-agent pattern library (successful trade patterns) - Feeds learning signals back to ensemble weight adjustment

Paul Orchestrator (`engine/agents/paul_orchestrator.py`)

Purpose: Full lifecycle management of GSD + Paul integration - Instantiates both plugins with per-agent configuration - Routes learning events from execution outcomes → plugins - Manages pattern library persistence (disk-backed) - Produces learning reports for the Platinum Report

LearningLoop (`engine/monitoring/learning_loop.py`)

Purpose: Closed-loop feedback from execution outcomes to all engines

7 Learning Channels

Channel	Feeds Back Into	Update Frequency
SIGNAL_ACCURACY	Engine tier weights	Per trade close
EXECUTION_QUALITY	Slippage model, routing	Per fill
REGIME_FEEDBACK	HMM transition priors	Daily
ALPHA_DECAY	AlphaOptimizer half-life	Per trade
RISK_CALIBRATION	Risk gate thresholds	Per risk event
AGENT_PERFORMANCE	Agent scorecard (promote/demote)	Weekly
CROSS_ASSET_FEEDBACK	Macro sector allocation	Weekly

Tier Weight Auto-Adjustment

```
# After 10+ signals per engine, LearningLoop adjusts ML ensemble tier weights:
accuracy_delta = rolling_accuracy - 0.50 # vs 50% baseline
weight_adj = clamp(accuracy_delta, -0.30, +0.30) # ±30% max
new_weight = default_weight × (1.0 + weight_adj)
```

Persistence

- Signal outcomes: `logs/learning_loop/outcomes_YYYYMMDD.jsonl`
- Learning snapshots: `logs/learning_loop/snapshot_YYYYMMDD_HHMMSS.json`
- GSD gradients: `logs/gsd_plugin/`
- Paul patterns: `logs/paul_plugin/`

LIVE RETURNS DASHBOARD + HOURLY CSV

HourlyRecapEngine (`engine/monitoring/hourly_recap.py`)

Purpose: Intraday portfolio monitoring with hourly snapshots + CSV export

Hourly CSV Output (`logs/returns/returns_YYYY-MM-DD.csv`)

Appended every hour (or on demand) during the trading day.

Column	Description
<code>timestamp</code>	ISO 8601 timestamp
<code>hour</code>	Hour of day (9–16)
<code>nav</code>	Current net asset value
<code>cash</code>	Available cash
<code>total_pnl</code>	Total P&L (realized + unrealized)
<code>realized_pnl</code>	Realized P&L from closed trades
<code>unrealized_pnl</code>	Mark-to-market unrealized
<code>daily_return_pct</code>	Return vs day-start NAV (%)
<code>gross_exposure</code>	Long + Short / NAV
<code>net_exposure</code>	Long - Short / NAV
<code>num_positions</code>	Active position count
<code>regime</code>	Current MetadronCube regime
<code>best_ticker</code>	Best performing position
<code>best_pnl</code>	Best position P&L
<code>worst_ticker</code>	Worst performing position
<code>worst_pnl</code>	Worst position P&L

Usage

```
from engine.monitoring.hourly_recap import export_hourly_csv
csv_path = export_hourly_csv(portfolio_state, positions)
```

Monitoring Stack

```
Hourly Snapshot → ASCII Recap (terminal)
                  → CSV Export (logs/returns/)
                  → Drift Alerts (position + sector)
                  → Vol Regime Tracking (LOW/NORMAL/ELEVATED/EXTREME)
                  → Risk Metrics (VaR, exposure, HHI, beta)
                  → Live Dashboard (Rich terminal, 550+ symbols)
                  → Learning Loop (feedback to engines)
```

LiveDashboard (`engine/monitoring/live_dashboard.py`)

Purpose: Real-time Rich-based terminal dashboard - 550+ symbol scanner (S&P 500 + key mid-caps) - Portfolio P&L, positions, NAV, cash - Signal activity feed (last 50 signals) - Sector heatmap (11 GICS sectors) - Risk metrics panel - Fallback: ASCII mode if Rich unavailable - Connected to IBKRBroker via callbacks

LiveEarningsGraph (`engine/monitoring/live_earnings_graph.py`)

Purpose: Real-time P&L curve visualization - Terminal-rendered P&L timeline - Hourly NAV progression - Drawdown tracking - Target hit markers (5% daily)

MONITORING & REPORTING

PlatinumReport — 30-section executive macro state (9 parts)

PlatinumReportV2 (`engine/monitoring/platinum_report_v2.py`) — Enhanced platinum report generator

PortfolioReport — Scenario engine + performance deep-dive (3 parts)

PortfolioAnalytics (`engine/monitoring/portfolio_analytics.py`) — Deep portfolio analytics + scenario engine

DailyReport — Open/close reports + sector heatmap

HeatmapEngine (`engine/monitoring/heatmap_engine.py`) — GICS sector heatmap visualization

SectorTracker — Sector performance + missed opportunities (>20% movers)

AnomalyDetector — Statistical anomaly scanner

MarketWrap — Narrative market summary

MemoryMonitor — Session tracking + EOD summary

WEB APPLICATION (`app/backend/`)

FastAPI-based web interface for the investment platform: - **REST API**: Portfolio state, signal pipeline, flow management - **SSE Streaming**: Real-time pipeline event streaming - **SQLAlchemy Models**: Flows, FlowRuns, Trades, Portfolio, API keys - **Pydantic Schemas**: Type-safe request/response validation - **Services**: Agent orchestration, graph visualization, backtesting

29 SIGNAL TYPES

MICRO_PRICE_BUY	MICRO_PRICE_SELL	RV_LONG	RV_SHORT
FALLEN_ANGEL_BUY	ML_AGENT_BUY	ML_AGENT_SELL	
DRL_AGENT_BUY	DRL_AGENT_SELL	TFT_BUY	TFT_SELL
MC_BUY	MC_SELL	QUALITY_BUY	QUALITY_SELL
SOCIAL_BULLISH	SOCIAL_BEARISH	SOCIAL_MOMENTUM	SOCIAL_REVERSAL
DISTRESS_FALLEN_ANGEL	DISTRESS_RECOVERY	DISTRESS_AVOID	
CVR_BUY	CVR_SELL		
EVENT_MERGER_ARB	EVENT_PEAD_LONG	EVENT_PEAD_SHORT	EVENT_CATALYST
HOLD			

INTELLIGENCE PLATFORM — 28 Reference Repos

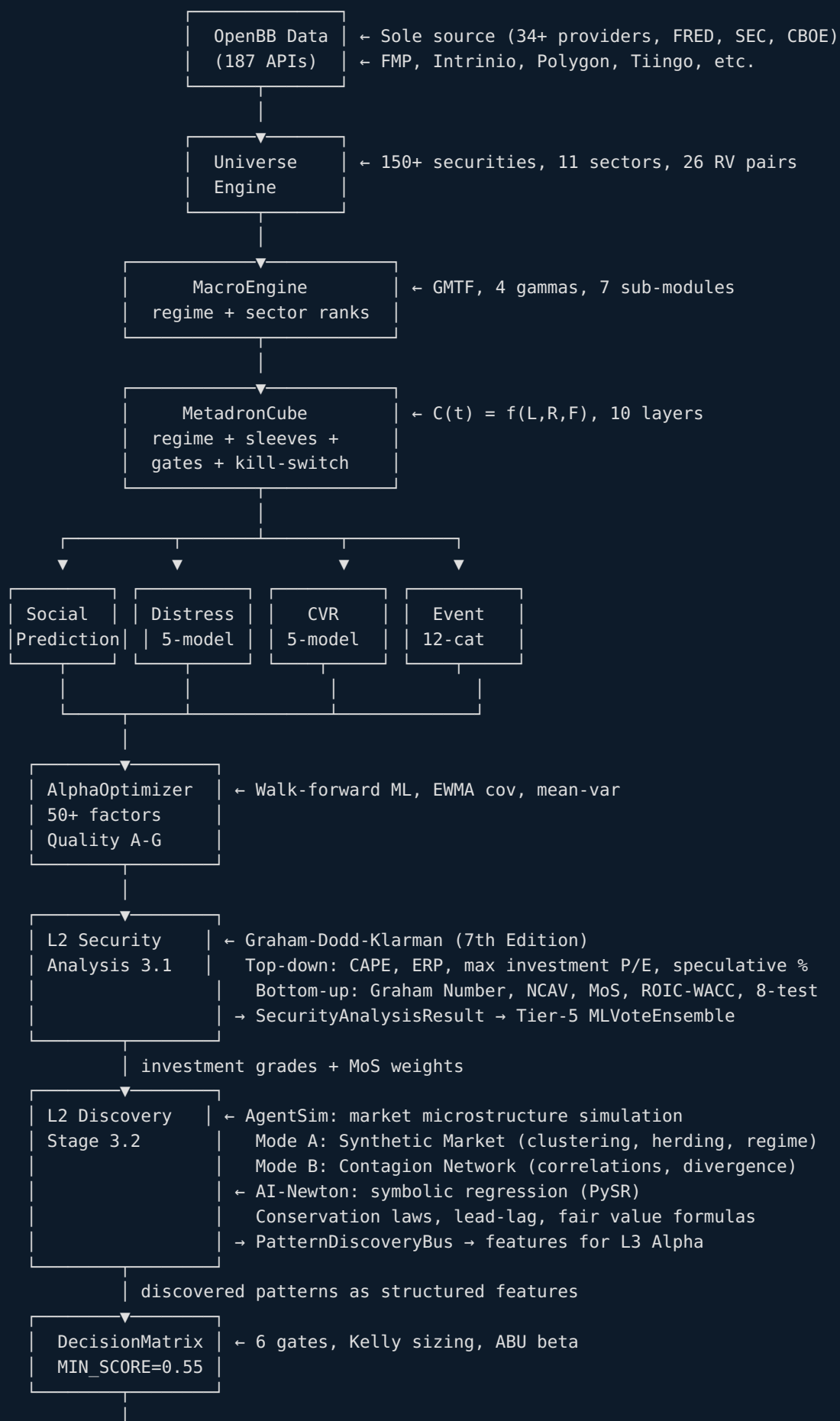
How Each Repo Maps to the Engine

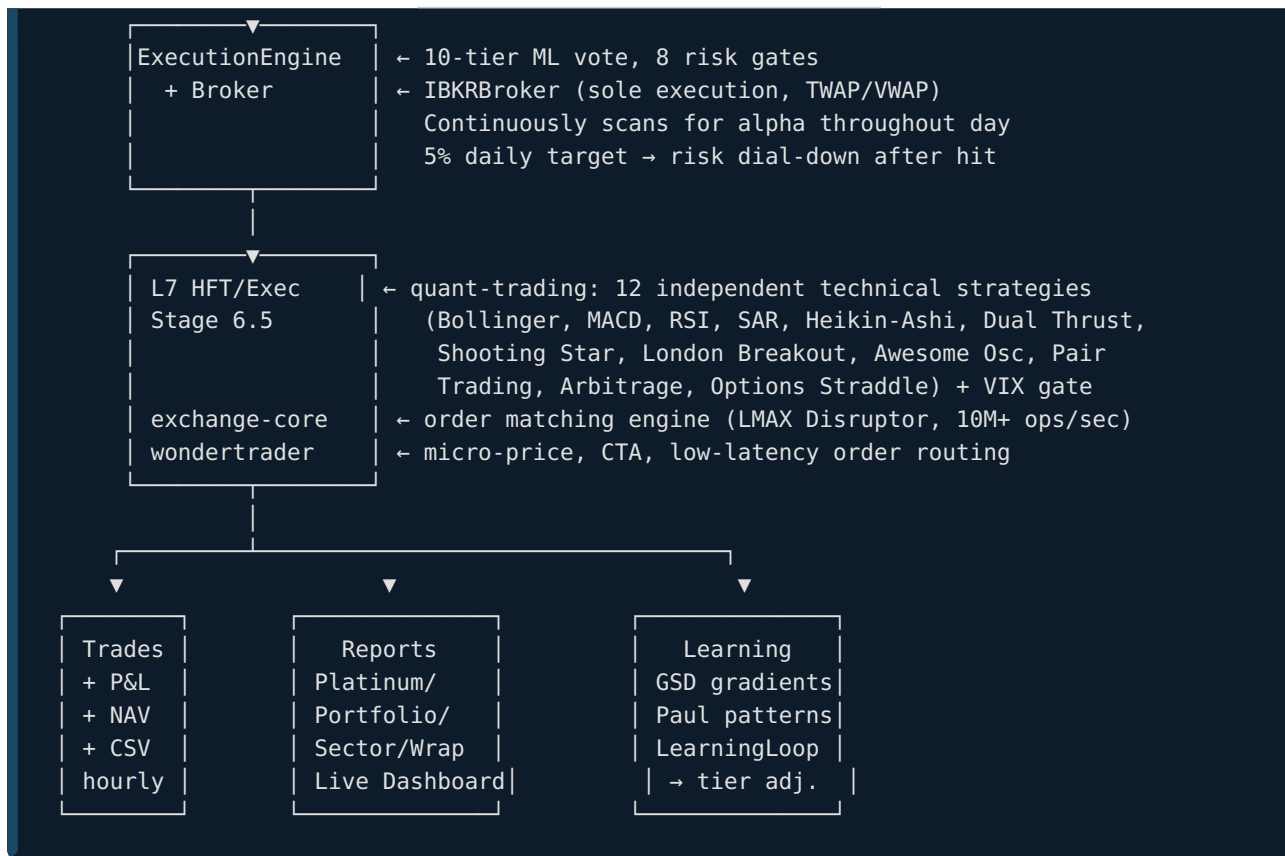
Repo	Layer	Feeds Into
Financial-Data	L1	OpenBB Data, UniverseEngine
open-bb	L1	MacroEngine, SectorRanker
hedgefund-tracker	L1	InstitutionalFlow signals
FRB	L1	DEPRECATED — FRED data now routed via OpenBB (openbb-fred provider). See open-bb
EquityLinkedGICPooling	L1	GIC pooling methodology
Quant-Developers-Resources	L1	Strategy templates library
Mav-Analysis	L2	Technical indicators, backtesting
quant-trading	L2+L7	Strategy library (Bollinger, Dual Thrust) + HFT Execution (12 technical strategies run independently in ExecutionEngine Stage 6.5)
stock-chain	L2	Chain analysis, flow decomposition
CTA-code	L2	Trend-following, momentum signals
TradeTheEvent	L2	EventDrivenEngine (BERT event detection)
QLIB	L3	AlphaOptimizer (factor mining, pipeline)
Stock-technical-prediction-model	L3	AlphaOptimizer (LSTM/CNN predictions)
Stock-prediction	L3	Additional prediction models
ML-Macro-Market	L3	MacroEngine (regime classification)
AI-Newton	L2+L4	Pattern Discovery (PySR symbolic regression: conservation laws, lead-lag, fair value) + Decision validation
ai-hedgefund	L4	Multi-agent portfolio, CVREngine
financial-distressed-repo	L4	DistressedAssetEngine (baseline)
sophisticated-distress-analysis	L4	DistressedAssetEngine (advanced)
FinancialDistressPrediction	L4	DistressedAssetEngine (GBM reference)
Kserve	L5	Model serving infrastructure
nvidia-repo	L5	GPU-accelerated training/inference
Air-LLM	L5	Efficient LLM inference
Ruflo-agents	L6	Agent orchestration framework

Repo	Layer	Feeds Into
MiroFish	L2+L6	Pattern Discovery (CAMEL-AI dual sim: clustering, herding, contagion, divergence) + Social prediction
exchange-core	L7	HFT Execution — Ultra-low-latency order matching engine (Python/ LMAX Disruptor ring buffer, 10M+ ops/sec concept)
wondertrader	L7	HFT Execution — CTA trend-following, micro-price engine, TWAP/ VWAP routing, multi-timeframe analysis
markov-model	L3	HMM Regime Detection — hmmlearn GaussianHMM for data-driven regime classification → MetadronCube RegimeEngine via MarkovRegimeBridge

DATA FLOW DIAGRAM







EXPECTED OUTPUT (Full Pipeline Run)

Morning Open generates:

1. **REGIME**: BULL/BEAR/TRANSITION/STRESS/CRASH + VIX + SPY returns
2. **SECTOR RANKINGS**: 11 sectors ranked by macro-adjusted composite score
3. **METADRON CUBE**: Regime, target beta, beta cap, max leverage, risk budget, 5 sleeve allocations
4. **ALPHA OPTIMIZER**: Expected return, volatility, Sharpe, top 5 signals with ticker/tier/alpha
5. **TRADES**: N trades executed (side, qty, ticker, vote_score)
6. **PORTFOLIO**: NAV, cash, P&L, positions, gross/net exposure
7. **CONTAGION**: 7 scenario systemic risk scores
8. **STAT ARB**: Pair status, z-scores, signals
9. **DISTRESSED ASSETS**: 5-model ensemble scores, fallen angels, recovery candidates
10. **CVR**: 5-model valuations, trade signals
11. **EVENT-DRIVEN**: Top event opportunities, Kelly-sized positions
12. **OPTIONS $\theta+\Gamma$** : Best OTM level, theta, gamma, score
13. **RESEARCH BOTS**: 11 sector intelligence reports + DNA hierarchy
14. **PLATINUM REPORT**: 30-section executive macro state
15. **PORTFOLIO REPORT**: Scenario engine + performance analytics

-
16. **SECTOR HEATMAP**: ASCII visual sector performance
 17. **MEMORY STATUS**: Session tracking

Evening Close generates:

1. **MARKET WRAP**: Narrative summary
 2. **MISSED OPPORTUNITIES**: >20% movers not captured
 3. **CONTAGION EOD**: Updated systemic risk
 4. **STAT ARB EOD**: Pair status update
 5. **ANOMALIES**: Statistical anomalies detected
 6. **AGENT SCORECARD**: Rankings + promotion/demotion
 7. **CONVICTION AUDIT**: Override audit trail
 8. **PLATINUM CLOSE**: 30-section with EOD stats
 9. **PORTFOLIO CLOSE**: Full performance deep-dive
 10. **MEMORY EOD**: Session summary + state persistence
-

KEY COMMANDS

```
# Full morning pipeline (paper mode)
python3 run_open.py

# Full morning pipeline (live IBKR)
IBKR_HOST=127.0.0.1 IBKR_PORT=7497 python3 run_open.py

# Evening reconciliation
python3 run_close.py

# Hourly recap
python3 run_hourly.py

# Platform health
python3 bootstrap.py

# Platform orchestrator status
python3 core/platform.py

# Run all tests (69 tests)
python3 -m pytest tests/ -v

# Verify all backends
python3 Installation-Back-end-Files/verify_install.py
```

COMPONENT INSTALLATION STATUS (14 backends)

COMPONENT	PIP	IMPORTS	INSTANTIATES	RUNS LIVE
OpenBB SDK	✓ 4.7.1	✓	✓	✓ API live
CAMEL-AI (MiroFish)	✓ 0.2.89	✓ guarded	✓	✓ numpy sim
PySR (AI-Newton)	✓ 1.5.9	✓ guarded	✓	✓ OLS fallback
PyTorch	✓ 2.10.0	✓ guarded	✓	✓ via backends
Transformers/FinBERT	✓ 5.3.0	✓ guarded	✓	✓ rule-based ¹
QLIB	✓ 0.9.8	✓	✓	✓ Alpha158 ²
Air-LLM	✓ 2.11.0	✓ guarded	✓	✓ rule-based
LightGBM	✓ 4.6.0	✓ guarded	✓	✓ via QLIB bk
XGBoost	✓ 3.2.0	✓ guarded	✓	✓ via QLIB bk
hmmlearn	✓ 0.3.3	✓	✓	✓ HMM fitted
quant-trading	✓ native	✓	✓	✓ standalone
Kserve	✓ 0.17.0	✓ SDK	✓	✓ local mode ³
exchange-core	✓ native	✓	✓	✓ matching eng
wondertrader	✓ native	✓	✓	✓ CTA+HFT
TOTALS:	14/14 ✓	14/14 ✓	14/14 ✓	14/14 ✓

¹ FinBERT catches proxy 403, falls back to 11-category rule-based classifier

² QLIB installed from source (editable), numpy Alpha158 28-factor fallback

³ Kserve local mode: register/predict/save/load work without K8s cluster

LOG & OUTPUT DIRECTORY STRUCTURE

```
logs/
├─ returns/           ← Hourly CSV: returns_YYYY-MM-DD.csv
├─ paper_broker/      ← Paper broker trade logs
├─ ibkr_broker/       ← IBKR order audit trail + algo execution logs
├─ learning_loop/     ← Signal outcomes JSONL + snapshots
├─ gsd_plugin/        ← Gradient Signal Dynamics logs
├─ paul_plugin/       ← Paul pattern library logs
├─ gsd_workflow/      ← GSD workflow traces
├─ agent_factory/     ← Agent creation logs
├─ agent_scorecard/   ← Agent ranking history
├─ backtest/          ← Backtesting results
├─ enforcement/      ← Risk gate enforcement logs
├─ ingestion/        ← Data ingestion logs
├─ platinum/         ← Platinum Report outputs
├─ portfolio/        ← Portfolio Report outputs
```

DESIGN RULES (IMMUTABLE)

1. **All data via OpenBB** (sole source, 34+ providers) — no yfinance dependency

2. **IBKR sole broker** — IBKRBroker via `ib_insync`, native TWAP/VWAP algo orders
3. **6-layer architecture is immutable** — extend within layers, not across
4. **Beta managed within 7–12% corridor** — vol-normalised
5. **Alpha targeted at 95%+** — aggressive multi-sleeve allocation
6. **All LLM agents use Anthropic API** (claude-opus-4-6)
7. **Pure-numpy fallbacks** — no bridge crashes if ML framework missing
8. **try/except on ALL external imports** — system runs degraded, never broken
9. **Tests must pass** before pushing
10. **Session continuity** — CLAUDE.md serves as bootstrap context
11. **Hourly CSV export** — every snapshot persisted to `logs/returns/` for audit
12. **Learning loop always active** — every trade outcome feeds back into tier weights

FULL SYSTEM INTEGRATION — PLATFORM ORCHESTRATOR

Pipeline Flow (Complete)

```
InvestmentPlatformOrchestrator (Master)
|
├─ Step 1: UniverseEngine ————— 1,044+ securities (SP500+400+600)
├─ Step 2: MacroEngine ————— Regime: BULL/BEAR/TRANSITION/STRESS/CRAS
H
├─ Step 3: MetadronCube ————— Intelligence tensor, sleeve allocation
├─ Step 4: SecurityAnalysisEngine ——— Graham-Dodd-Klarman (L2/L2.5)
├─ Step 5: Signal Engines (parallel)
|   ├── ContagionEngine ————— Cross-asset contagion
|   ├── StatArbEngine ————— Mean reversion + cointegration
|   ├── SocialPredictionEngine ——— AgentSim market microstructure
|   ├── DistressedAssetEngine ——— 5-model distress ensemble
|   ├── CVREngine ————— Contingent value rights
|   ├── EventDrivenEngine ——— Merger arb, PEAD, catalysts
|   ├── FedLiquidityPlumbing ——— Fed balance sheet, money velocity
|   └─ AgentSimEngine ————— AgentSim market microstructure
├─ Step 6: PatternRecognitionEngine ——— Chart patterns, ML anomalies
├─ Step 7: AlphaOptimizer ————— Walk-forward ML + mean-variance
├─ Step 8: BetaCorridor ————— Beta management (7-12% corridor)
├─ Step 9: MonteCarloRiskEngine ——— 5,000 sims, VaR, CVaR, stress
├─ Step 10: OptionsEngine + BlackScholes — Greeks, IV, mispricing scanner
├─ Step 11: Internal Analysis Pipeline — Technical + Fundamental + Macro
├─ Step 12: Trade Thesis Generation ——— Scored trade ideas
└─ Step 13: ExecutionEngine → L7 → IBKRBroker — Order routing and execution
```

Data Flow

```

Market Hours (09:30-16:00 ET):
  IBKR (real-time quotes, 2-5s) → Signal Pipeline → L7 → IBKRBroker (execution)
  Heartbeat: 2-min cadence, open/close bursts at 1-min

After Market Close (16:00-20:00 ET):
  OpenBB (historical + FRED + macro) → Backtesting → Model Retrain → Pattern Learning
  30-min cadence for earnings reactions

Overnight:
  OpenBB continuous backtesting → Model enhancement → Agent skill deployment

```

Options Trading Flow

```

BlackScholesEngine
├─ Theoretical pricing (Call + Put)
├─ Full Greeks (Delta, Gamma, Theta, Vega, Rho)
├─ Implied volatility solver (Brent's method)
├─ Monte Carlo pricing (10,000 GBM paths)
├─ Mispricing scanner (theoretical vs market)

MonteCarloRiskEngine
├─ 1,000 MC paths per ticker (21-day horizon)
├─ VaR 95/99, CVaR 95/99
├─ Stress VaR (2x volatility)
├─ Tail risk scoring

OptionsEngine
├─ Receives BlackScholes theoretical prices
├─ Identifies mispriced options (>10% threshold)
├─ Computes hedge ratios via Greeks
├─ Routes to L7 → IBKRBroker for execution

```

Broker Hierarchy

```

IBKRBroker      → SOLE: Equities + Options + Futures (TWS/Gateway, TWAP/VWAP)
Trade Log       → RECON: Records all generated orders vs broker fills
PaperBroker     → BACKTESTING ONLY: Historical simulation (never used for live)

```

Fixed Income Engine & Router

FixedIncomeEngine (`engine/signals/fixed_income_engine.py`)

Aggregates fixed income data from existing data sources. No new external providers — composes data from OpenBB/FRED, MacroEngine, and broker positions.

Data Flow:

```

OpenBB/FRED (treasury rates, credit spreads)
  |
  ▼
FixedIncomeEngine ← MacroEngine (yield curve analysis, credit pulse)
                    ← Broker (FI positions: TLT, IEF, SHY, LQD, HYG, etc.)
  |
  ▼
/api/engine/fixed-income router (6 endpoints)
  |
  ▼
Tab 18 (Fixed Income) – useEngineQuery hooks

```

Methods: | Method | Data Source | Returns | |---|---|---| | `get_summary()` | `get_treasury_rates()` + `MacroEngine.get_yield_curve_analysis()` | Portfolio summary: avg yield, duration, rating, DV01 | | `get_holdings()` | Broker positions filtered for FI ETFs | List of FI holdings with P&L | | `get_yield_curve()` | `get_fred_series()` for 11 tenors (1M through 30Y) | `[{tenor, rate, change_ld}]` | | `get_credit_quality()` | `get_credit_spreads()` (BAMLH0A0HYM2, BAMLC0A4CBBB) | Market credit quality distribution | | `get_duration_ladder()` | `get_fred_series()` grouped by maturity bucket | `[{bucket, avg_rate}]` | | `get_spread_history(days)` | `get_credit_spreads()` historical | `{dates, ig_spread, hy_spread}` |

Fixed Income Router (`engine/api/routers/fixed_income.py`)

Mount: `/api/engine/fixed-income`

Endpoint	Method	Description
<code>/summary</code>	GET	Portfolio-level FI summary
<code>/holdings</code>	GET	Bond holdings from broker
<code>/yield-curve</code>	GET	Full yield curve (11 tenors)
<code>/credit-quality</code>	GET	Credit quality distribution
<code>/duration-ladder</code>	GET	Duration/maturity buckets
<code>/spread-history?days=90</code>	GET	Historical IG/HY spreads

Macro Router — Historical Time Series Extensions

Added to existing `/api/engine/macro` router:

Endpoint	FRED Series	Description
<code>/spread-history</code>	DGS10 - DGS2	2s10s yield spread (30 days)
<code>/vix-history</code>	VIXCLS	VIX time series (30 days)
<code>/dxy-history</code>	DTWEXBGS	Dollar index (30 days)

Dashboard Tab Status

Tab	Status	Data Source
Tab 18 (Fixed Income)	WIRED TO LIVE DATA	<code>/fixed-income/*</code> + <code>/macro/yield-curve</code> + <code>/macro/credit-pulse</code>
Tab 19 (Macro)	WIRED TO LIVE DATA	<code>/macro/*</code> + FRED historical time series
Tab 25 (Money Velocity)	UNTOUCHED	As-is per user instruction
Tab 26 (Thinking)	LIVE SSE STREAM	<code>/api/allocation/scan/thinking</code> SSE + <code>/api/allocation/scan/status</code>
Tab 27 (Collateral/Margin)	WIRED TO LIVE DATA	<code>/api/allocation/collateral/status</code>

ALLOCATION ENGINE MODULE (engine/allocation/)

Architecture Overview



AllocationRules Dataclass

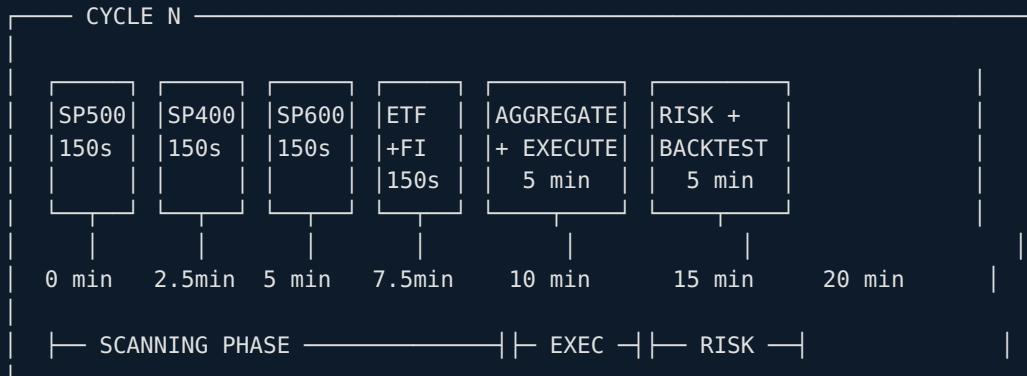
Parameter	Default	Description
max_drawdown_kill_switch	0.20	20% drawdown → HALT
ig_equity_pct	0.40	IG equities (40%)
hy_equity_pct	0.10	HY equities (10%)
distressed_equity_pct	0.10	Distressed equity (10%)
tltw_cashflow_pct	0.15	DIV/Cashflow ETFs (15%)
fi_macro_pct	0.05	FI + Macro RV (5%)
event_driven_cvr_pct	0.10	Event-driven / CVR (10%)
options_notional_pct	0.25	Options notional (25%: IG 10%, HY 10%, Distressed 5%)
futures_beta_pct	0.15	Futures beta hedge (15%)
margin_pct	0.08	Margin real capital (8%)
money_market_pct	0.02	Cash / dry powder (2%)
profit_take_threshold	0.20	20% P&L → liquidate overlays
drip_rule	True	ETF distributions must be reinvested
alpha_primary_goal	True	Alpha extraction is the primary goal

Bucket Classification Logic

```
Signal → classify_opportunity():
  OPTION → OPTIONS_IG / OPTIONS_HY / OPTIONS_DISTRESSED (by signal type)
  FUTURE/DERIVATIVE → MARGIN
  FIXED_INCOME → FI_MACRO or EVENT_DRIVEN_CVR
  ETF → DIV_CASHFLOW ETF
  EQUITY + DISTRESS signal → HY_DISTRESSED
  EQUITY + EVENT signal → EVENT_DRIVEN_CVR
  EQUITY default → IG_EQUITY
```

Full Universe Scan Timing Diagram

Market Hours: 09:30 – 16:00 ET
 Cycle Duration: 20 minutes
 Cycles Per Hour: 3

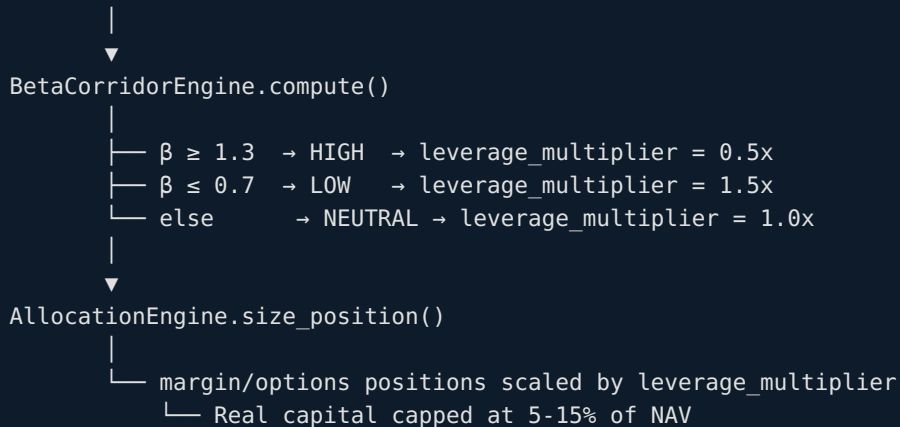


Per-run output:

- Favored equity names + signal scores
- Options/derivatives plays
- ETF/FI positions (Run 4 emphasis)
- Confidence scores + regime context
- JSON → AllocationEngine → L7 Surface

Beta Corridor → Leverage → Margin Bucket Flow

Portfolio Returns + SPY Returns



Kill Switch Flow

```

Current NAV → KillSwitchMonitor.check(nav, hwm)
├── Update high_water_mark = max(hwm, nav)
├── drawdown = (hwm - nav) / hwm
├── drawdown < 20% → CLEAR → normal operations
└── drawdown ≥ 20% → TRIGGERED
    ├── Log event with timestamp
    ├── All new orders HALTED
    ├── Notify operator
    └── Await manual reset()
  
```

L7 Execution Surface Integration

```

AllocationSlate (from aggregate_runs)
└── ▼
    validate_against_kill_switch()
    ├── HALTED → empty slate, no execution
    └── CLEAR → proceed
        └── ▼
            L7UnifiedExecutionSurface
            ├── Product classification (equity/option/future)
            ├── Risk gates (8 gates)
            ├── DecisionMatrix (6 gates)
            └── Broker execution (IBKR via L7)
  
```

Thinking Tab Data Flow (SSE Pipeline)

```

FullUniverseScan.run_universe()
├── Per-ticker analysis → signal discovery
│   └── SignalEventBus.emit({
│       type: "signal_discovered",
│       ticker, signal_type, confidence,
│       alpha_score, regime_context, bucket
│   })
└── SignalEventBus → asyncio.Queue (per subscriber)
    └── FastAPI SSE endpoint: /api/allocation/scan/thinking
        └── Express proxy: /api/allocation/scan/thinking
            └── React EventSource → ThinkingTab.tsx
                ├── SignalCard rendering (color-coded by bucket)
                ├── Progress bars (per universe run)
                ├── Cycle phase indicator
                ├── Kill switch status badge
                └── Beta corridor indicator
  
```

Collateral Tab Data Flow

```

AllocationEngine._utilization → collateral/status endpoint
├── OPTIONS_IG + OPTIONS_HY + OPTIONS_DISTRESSED → options_premium
├── MARGIN → futures_margin
├── BetaCorridorEngine.status() → corridor + leverage
├── KillSwitchMonitor.status() → drawdown + trigger
└── total_real_capital = margin + options * 0.3
    └── FastAPI: GET /api/allocation/collateral/status
        └── Express proxy → React CollateralTab.tsx
            ├── UtilizationGauge (0-15% range, alert at >12%)
            ├── Beta corridor card
            ├── Notional exposure breakdown (IG/HY/Distressed/Futures)
            ├── Kill switch badge
            └── History chart (margin utilization over session)
  
```

S&P 1500 Universe Composition

Universe	Source	Ticker Count	Run
S&P 500	cross_asset_universe.py SP500_TICKERS	~500	Run 1
S&P 400 MidCap	cross_asset_universe.py SP400_TICKERS	~400	Run 2
S&P 600 SmallCap	cross_asset_universe.py SP600_TICKERS	~600	Run 3
ETF + Fixed Income	sp1500_universe.py ETF_TICKERS + FI_TICKERS	~70	Run 4

ETF list: TLTW, QQQ, SPY, IWM, HYG, LQD, TLT, PDBC, GLD, USO + 11 GICS sector ETFs + factor/income ETFs
 FI instruments: TLT, IEF, SHY, HYG, LQD, EMB, BKLN, AGG, BND, VCIT, VCSH, MUB, TIP, GOVT + international FI

New API Endpoints

Method	Path	Description
GET	<code>/api/allocation/rules</code>	Current allocation rules as JSON
POST	<code>/api/allocation/rules</code>	Operator rule updates (with timestamp logging)
GET	<code>/api/allocation/status</code>	Bucket utilization, kill switch, beta corridor
GET	<code>/api/allocation/slate</code>	Last computed allocation slate
GET	<code>/api/allocation/scan/status</code>	Current scan cycle status (run, elapsed, phase)
GET	<code>/api/allocation/scan/thinking</code>	SSE stream of real-time scan signals
GET	<code>/api/allocation/collateral/status</code>	Margin bucket: real capital, notional, leverage

All endpoints are registered in FastAPI at `/api/allocation/*` and proxied through Express.

Backtest Integration

All allocation rules apply identically in backtest mode: - `AllocationEngine(backtest=True)` or `apply_rules(opportunity_list, backtest=True)` - Kill switch applied historically (20% drawdown stops the test) - Beta corridor computed from historical returns - Bucket constraints enforced on historical positions - DRIP rule simulated (distribution reinvestment logged) - Margin bucket modeled at 5-15% real capital - FullUniverseScan with `backtest=True` fast-forwards timing (no real 150s waits)

New Frontend Tabs

#	Tab	Path	Component	Description
26	THINKING	/thinking	ThinkingTab	Live SSE scan reasoning, signal cards, kill switch, beta corridor
27	MARGIN	/collateral	CollateralTab	Margin bucket, utilization gauge, notional breakdown, P&L
28	CHAT	/chat	ChatTab	NanoClaw operator chat + Ruflo swarm agent interface

Reporting — Allocation Rules Section

The Reporting tab (/reports) now includes a collapsible "ALLOCATION RULES IN EFFECT" section showing: - Date-stamped active rules with bucket targets and current utilization - Options notional breakdown (IG/HY/Distressed) - Kill switch status - Beta corridor summary - DRIP reinvestment event count - Operator rule change count - Universe scan cycle summary (cycles completed, signals, phase)

Agent Governance & Chat System

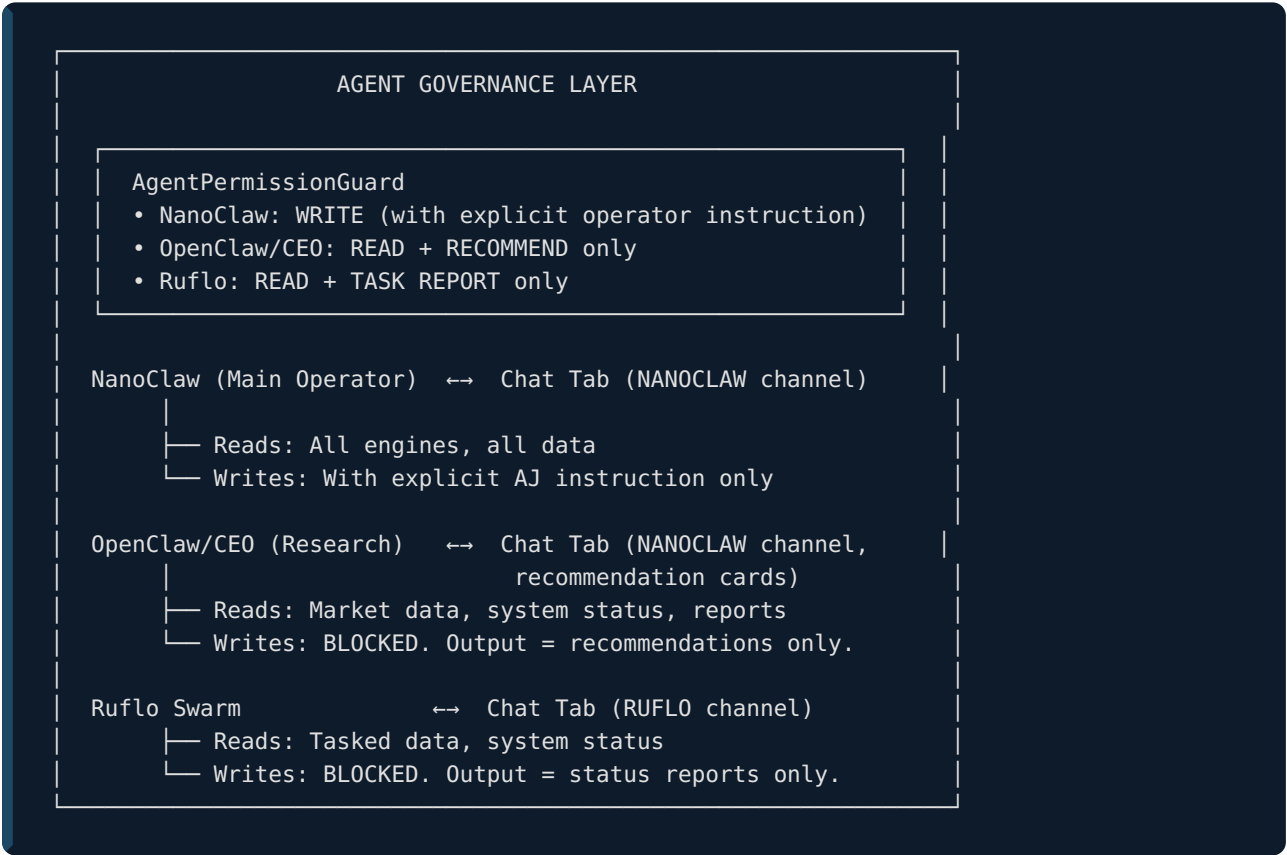
Agent Identities

Agent	ID	Role	Permission Level
NanoClaw	nanoclaw	Main Operator Agent	WRITE (with explicit operator instruction)
OpenClaw/CEO	openclaw	CEO Research Agent	READ + RECOMMEND only
Ruflo	ruflo	Swarm Task Agents	READ + TASK REPORT only

Permission Matrix

Action	NanoClaw	OpenClaw/CEO	Ruflo
read_data	Yes	Yes	Yes
search	Yes	Yes	Yes
analyze	Yes	Yes	Yes
recommend	Yes	Yes	Yes
write_file	Yes (with permission)	BLOCKED	BLOCKED
execute_trade	Yes (with permission)	BLOCKED	BLOCKED
modify_config	Yes (with permission)	BLOCKED	BLOCKED
update_rules	Yes (with permission)	BLOCKED	BLOCKED
push_code	Yes (with permission)	BLOCKED	BLOCKED
call_l7	Yes (with permission)	BLOCKED	BLOCKED

Chat Tab Architecture



NanoClaw Write-Gate Protocol

1. Default mode: READ + RECOMMEND
2. To execute a write action, the operator (AJ) must explicitly instruct (e.g., "go ahead", "execute", "apply this")
3. NanoClaw always confirms before writing: "I'll [action]. Shall I proceed?"
4. Permission enforced server-side by `AgentPermissionGuard.assert_allowed(action, agent_id)`

CEO Recommendation Approval Workflow

1. OpenClaw/CEO produces a research recommendation
2. Recommendation appears as an amber-bordered card in the NanoClaw chat channel
3. Card includes [APPROVE] and [DISMISS] buttons
4. APPROVE routes the recommendation to NanoClaw for execution (still requires NanoClaw confirmation)
5. DISMISS removes the recommendation from the queue

API Endpoints

Endpoint	Method	Description
<code>/api/chat/nanoclaw/messages</code>	GET	NanoClaw message history
<code>/api/chat/nanoclaw/send</code>	POST	Send message to NanoClaw (SSE stream response)
<code>/api/chat/nanoclaw/stream</code>	GET	SSE stream for NanoClaw status updates
<code>/api/chat/ruflo/agents</code>	GET	List active Ruflo agents + status
<code>/api/chat/ruflo/send</code>	POST	Send message to Ruflo agent or broadcast
<code>/api/chat/recommendations</code>	GET	Pending CEO recommendations
<code>/api/chat/recommendations/{id}/approve</code>	POST	Approve a recommendation
<code>/api/chat/recommendations/{id}/dismiss</code>	POST	Dismiss a recommendation
<code>/api/chat/agent-permissions</code>	GET	Permission map for all agents

Files Added

```
engine/agents/nanoclaw/
├─ __init__.py
├─ permission_guard.py      # AgentPermissionGuard class + AGENT_PERMISSIONS dict
├─ agent_config.py         # Agent identity constants + display metadata
├─ nanoclaw_agent.py       # NanoClawAgent – streaming Claude API wrapper
├─ groups/
│   ├─ main/CLAUDE.md      # NanoClaw operator memory
│   ├─ openclaw/CLAUDE.md  # OpenClaw/CEO read-only restrictions
│   └─ ruflo/CLAUDE.md     # Ruflo swarm group memory
engine/api/routers/chat.py  # FastAPI chat router (/api/chat/*)
client/src/pages/chat-tab.tsx # React chat tab (NANOCLAW + RUFLO sub-tabs)
```

Tab Groups

The frontend tab navigation is organized into 7 groups. Each group contains related tabs displayed under a group header in the dropdown selector.

Group	Tabs
CORE	LIVE, WRAP, OPENBB, VELOCITY, CUBE
TRANSACTIONS	ALLOC, THINKING, RISK, MARGIN, RECON
PRODUCTS	ETF, MACRO, FIXED INC, FUTURES
AGENTS	AGENTS, CHAT, TECH
ANALYSIS	STRAT, QUANT, ARB, BACKTEST
SIMULATION	MC SIM, SIM, ML, ML MODELS
REPORTING	TRANSACTION LOG, TCA, REPORTS, ARCHIVE

Defined in `client/src/App.tsx` as `TAB_GROUPS`. A flat `ALL_TABS` array is derived via `flatMap` for backward compatibility with pinning, routing, and navigation logic.

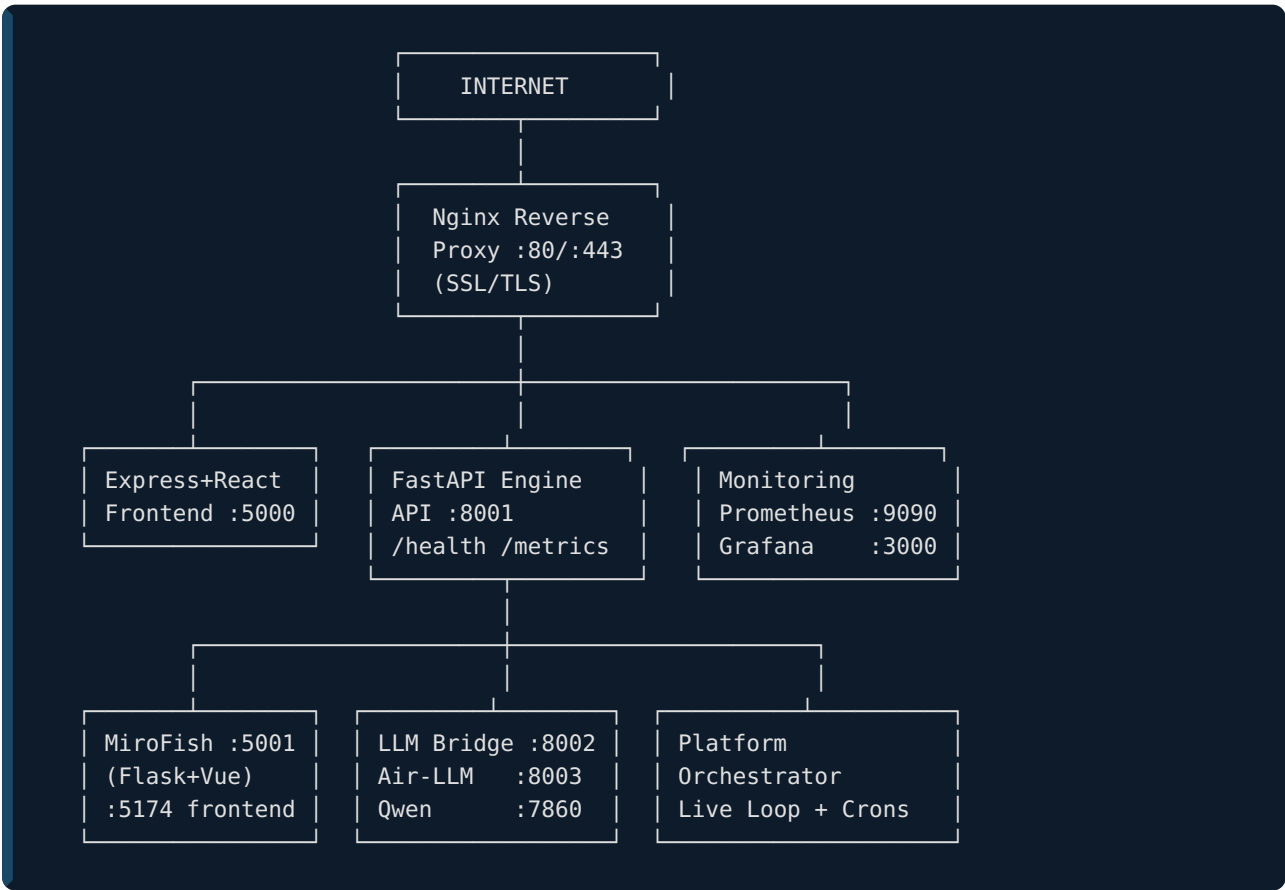
Deployment & Infrastructure

Contabo VPS Specification

Component	Recommended
CPU	12+ vCPU (AMD EPYC)
RAM	64 GB
Disk	800 GB NVMe
Network	400+ Mbit/s
OS	Ubuntu 22.04 LTS

For GPU inference (Qwen 2.5-7B, Air-LLM): add Contabo GPU VPS or external GPU server.

Network Topology



Port Map

Service	Port	Protocol	External
Express + React Frontend	5000	HTTP	Via Nginx :443 /
FastAPI Engine API	8001	HTTP	Via Nginx :443 /api/engine/
Prometheus	9090	HTTP	Direct (restrict in prod)
Grafana	3000	HTTP	Direct or via Nginx
MiroFish Backend	5001	HTTP	Internal only
MiroFish Frontend	5174	HTTP	Internal only
LLM Inference Bridge	8002	HTTP	Internal only
Air-LLM Model Server	8003	HTTP	Internal only
Qwen 2.5-7B Server	7860	HTTP	Internal only
Redis	6379	TCP	Internal only

Service Dependency Tree

```
redis (standalone)
prometheus (standalone)
node-exporter (standalone)
└─ backend (FastAPI) ← depends on redis (optional)
    ├── frontend (Express+React) ← depends on backend health
    ├── mirofish-backend (Flask)
    ├── llm-inference-bridge ← depends on backend
    ├── airllm-model-server
    ├── ainewton-service
    ├── metadron-cube (24/7 regime)
    ├── live-loop (09:30-16:00 ET)
    ├── learning-loop (continuous ML)
    ├── platform-orchestrator
    ├── market-open (cron 09:30 M-F)
    ├── market-close (cron 16:00 M-F)
    └─ hourly-tasks (cron hourly)
grafana ← depends on prometheus health
nginx ← depends on frontend + backend + grafana
```

Docker Compose Services

Service	Image/Build	Port	Health Check
frontend	Node 20 (build from repo)	5000	curl http://localhost:5000/
backend	Python 3.11 (build from repo)	8001	curl http://localhost:8001/health
prometheus	prom/prometheus:v2.51.0	9090	wget http://localhost:9090/-/healthy
grafana	grafana/grafana:10.4.0	3000	wget http://localhost:3000/api/health
redis	redis:7-alpine	6379	redis-cli ping
node-exporter	prom/node-exporter:v1.7.0	9100	—
nginx	nginx:1.25-alpine	80, 443	curl http://localhost/

Volumes: prometheus-data , grafana-data , redis-data (persistent Docker volumes).

Prometheus Scrape Targets

Job	Target	Interval	Metrics
prometheus	localhost:9090	15s	Self-monitoring
metadron-engine-api	localhost:8001/metrics	10s	100+ metrics: portfolio, cube, trades, agents, TCA, macro
metadron-llm-bridge	localhost:8002/metrics	30s	LLM request counts, latency by backend
metadron-airllm	localhost:8003/metrics	30s	Air-LLM model inference
node-exporter	localhost:9100	15s	Host CPU, RAM, disk, network
metadron-frontend	localhost:5000/metrics	30s	Express request metrics

Alert rules: KillSwitchActive, DrawdownExceeded (>15%), DailyLossCircuitBreaker (>3% NAV), ScanCycleSlow (>300s), EngineAPIDown, AgentPermissionBlocksHigh (>10/min), VaRBreach (>\$300K), CreditStressElevated (HY OAS >600bps).

Retention: 30 days / 10 GB cap.

Grafana Dashboard Folders

Folder	Dashboards
Core	Platform Overview, Signal Pipeline, Risk Dashboard
Allocation	Portfolio Allocation, Cube Regime, Decision Matrix
Agents	Agent Scorecard, Agent Herding, NanoClaw Permissions
Velocity	Macro Indicators, Money Velocity, Fixed Income
Monitoring	Infrastructure, TCA Execution, LLM Inference, Data Sources

Environment Variables

ANTHROPIC_API_KEY , IBKR_HOST , IBKR_PORT , IBKR_CLIENT_ID , IBKR_PAPER_TRADE ,
OPENBB_TOKEN , ZEP_API_KEY , ENGINE_API_PORT , PORT , NODE_ENV , FLASK_PORT ,
LLM_BRIDGE_PORT , AIRLLM_PORT , QWEN_MODEL_PATH , QWEN_SERVER_PORT , QWEN_GPU_DEVICES ,
AIRLLM_MODEL_PATH , AIRLLM_GPU_DEVICES , ANTHROPIC_MODEL , LLM_BRIDGE_URL ,
METADRON_MODE , METADRON_CUBE_MODE , PYTHONUNBUFFERED , GRAFANA_ADMIN_PASSWORD

Startup Sequence

1. redis ← Cache/pub-sub
2. prometheus ← Metrics collection
3. node-exporter ← Host metrics
4. backend (FastAPI) ← Engine API + /metrics
5. frontend (Express) ← React SPA + proxy
6. grafana ← Visualization
7. nginx ← Reverse proxy + SSL
8. PM2 services: ← mirofish, llm-bridge, live-loop, cube, learning-loop, etc.

Key Health Check URLs

Service	URL	Expected
Engine API	http://localhost:8001/health	{"status": "ok"}
Frontend	http://localhost:5000/	200 OK
Prometheus	http://localhost:9090/-/healthy	"Healthy"
Grafana	http://localhost:3000/api/health	{"database": "ok"}
Redis	redis-cli ping	PONG

Full deployment guide: see `DEPLOYMENT.md` at repository root.